Dynamic Optimization as a Tool for Motion Planning and Control

TSFS12: Autonomous Vehicles – Planning, Control, and Learning Systems

Lecture 5: Erik Frisk <erik.frisk@liu.se>

LINKÖPING UNIVERSITY

Purpose of this Lecture

- Give a background on dynamic optimization, particularly with respect to applications in motion planning and control and the main ideas of the methods used:
  - motivation and challenges,
  - fundamental concepts and methods,
  - application examples.
- Demonstrate a case-study for optimization of motion primitives.
  - Connected to hand-in exercise 2.
- Model Predictive Control
  - Optimal path following, multi-vehicle collision avoidance, …
  - Connected to MPC lecture, hand-in exercise 3, extra assignment.
- Optimization is a good language for engineering tasks
  - merging control and learning — especially for dynamic systems

Autonomous racing @ Stanford

- Optimality
- Dynamic behavior
- Real-time

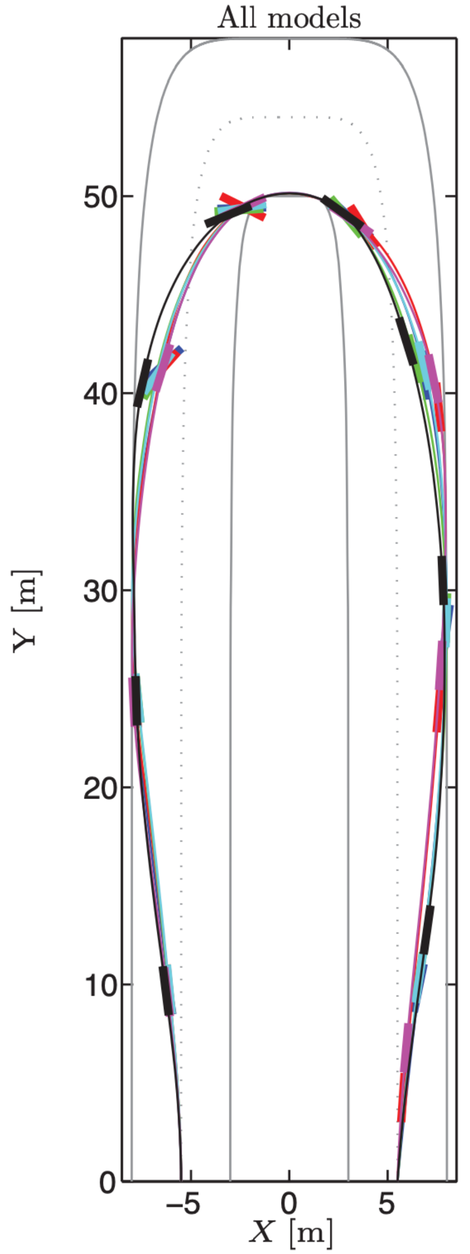
https://www.youtube.com/watch?v=joIsgP9StAY

Dynamic Optimization – Background

- In this lecture the focus is on numerical methods for dynamic optimization, since many real-world problems are intractable with analytical methods.
- Examples:
  - Motion planning for autonomous vehicles and robots,
  - Optimal battery-charging planning for autonomous electric vehicles,
  - Efficient electric motor and engine control,
  - Traffic-flow optimization in city environments.
  - …

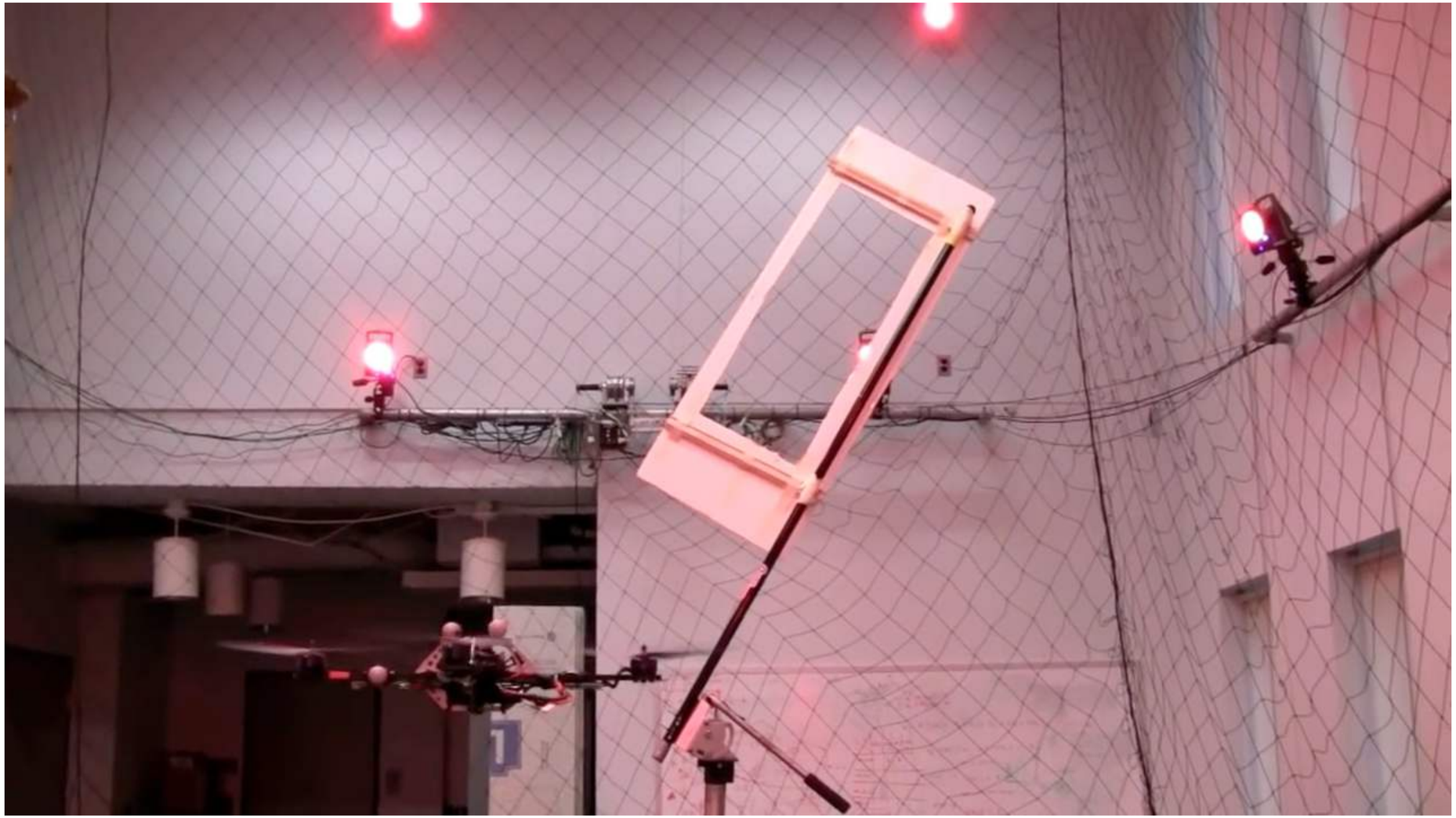
Dynamic Optimization – Examples

- Vehicle maneuvers at-the-limit of tire friction for development of future safety systems for autonomous vehicles.
- Traverse the hairpin turn in as short time as possible.



K. Berntorp, B. Olofsson, K. Lundahl, & L. Nielsen: ”*Models and methodology for optimal trajectory generation in safety-critical road-vehicle manoeuvres*”, Vehicle System Dynamics, 52(10):1304–1332, 2014.

Dynamic flight



https://www.youtube.com/watch?v=MvRTALJp8DM

Context in the Architecture for an Autonomous Vehicle

Dynamic optimization is possible to apply at all layers in the architecture for different tasks

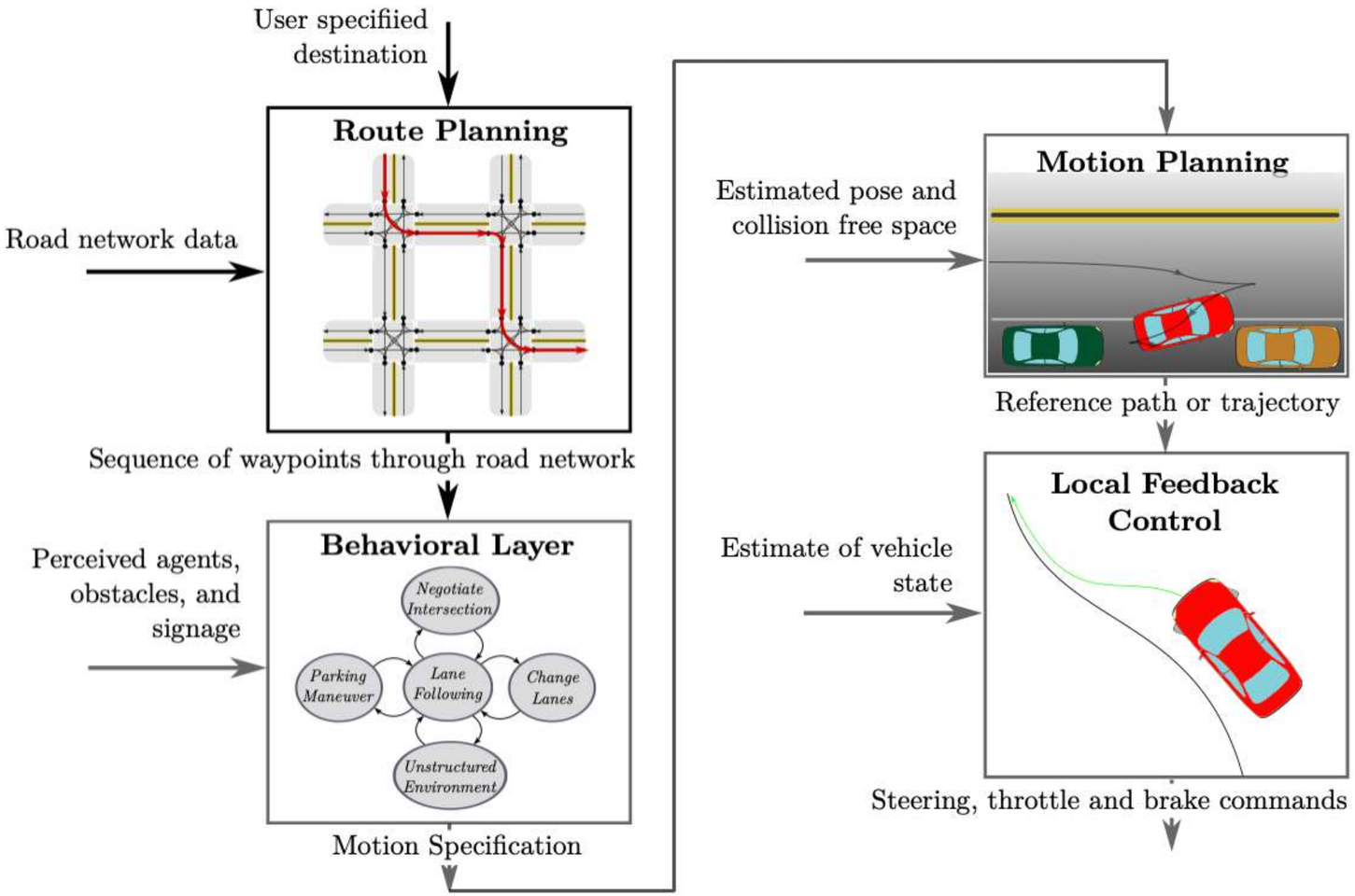


Figure from: Paden, B., Čáp, M., Yong, S. Z., Yershov, D., & Frazzoli, E: ”*A survey of motion planning and control techniques for self-driving urban vehicles*”. IEEE Transactions on Intelligent Vehicles, 1(1), 33-55, 2016.

Expected Take-Aways from this Lecture

- Be familiar with basic concepts in optimization and common methods for numerical optimal control for systems described by continuous-time dynamic models.
- Not expected to get insights into all details of the treated methods, primarily on a general level for understanding of their applicability.
- Introduce, but no in-depth explanations, of some key notions
- Have knowledge about different applications of dynamic optimization for motion planning and control for autonomous vehicles.
- Basic for the more detailed follow-up lecture at the end of the course on Optimal Collision Avoidance

Literature Reading

The following book and article sections are the main reading material for this lecture. References to further reading are provided throughout the slides and at the end of the lecture slides.

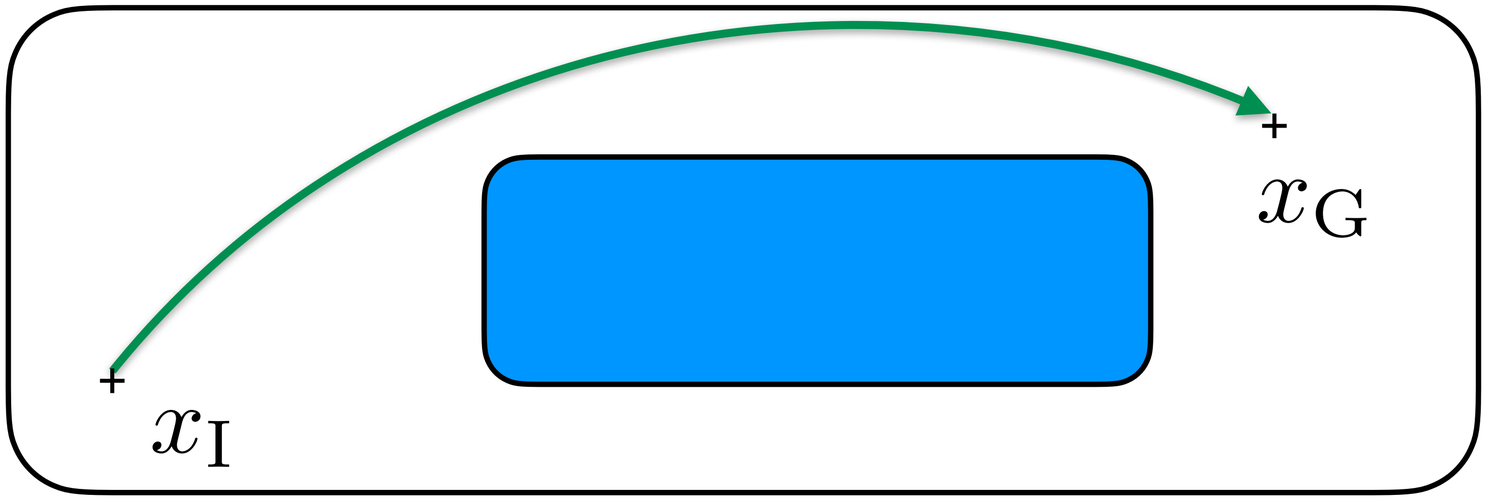
- Limebeer, D. J., & A. V. Rao: ”*Faster, higher, and greener: Vehicular optimal control*”. IEEE Control Systems Magazine, 35(2), 36-56, 2015.
- For a more mathematical treatment of the topic of numerical optimal control and further reading on the methods presented in this lecture:

  Chapter 8 in Rawlings, J. B., D. Q. Mayne, & M. Diehl: “*Model Predictive Control: Theory, Computation, and Design*”, 2nd Edition. Nob Hill Publishing, 2017.

Formulating Dynamic Optimization Problems

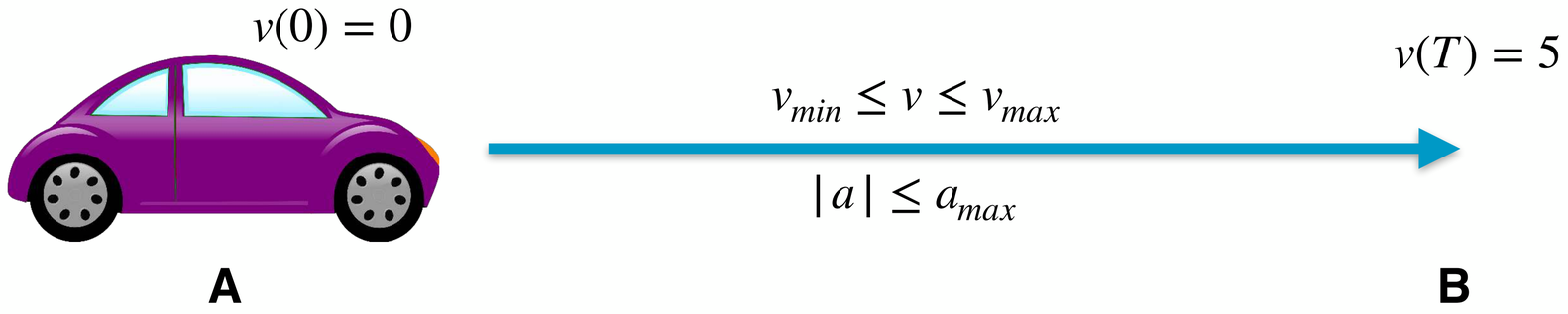
Recall Motion-Planning Problem from Lecture 4

- Compute a strategy for transferring a vehicle from an initial state to another desired state.
- Constraints on control inputs and states, as well as a performance criterion to be fulfilled.



A First Optimization Example (1/3)

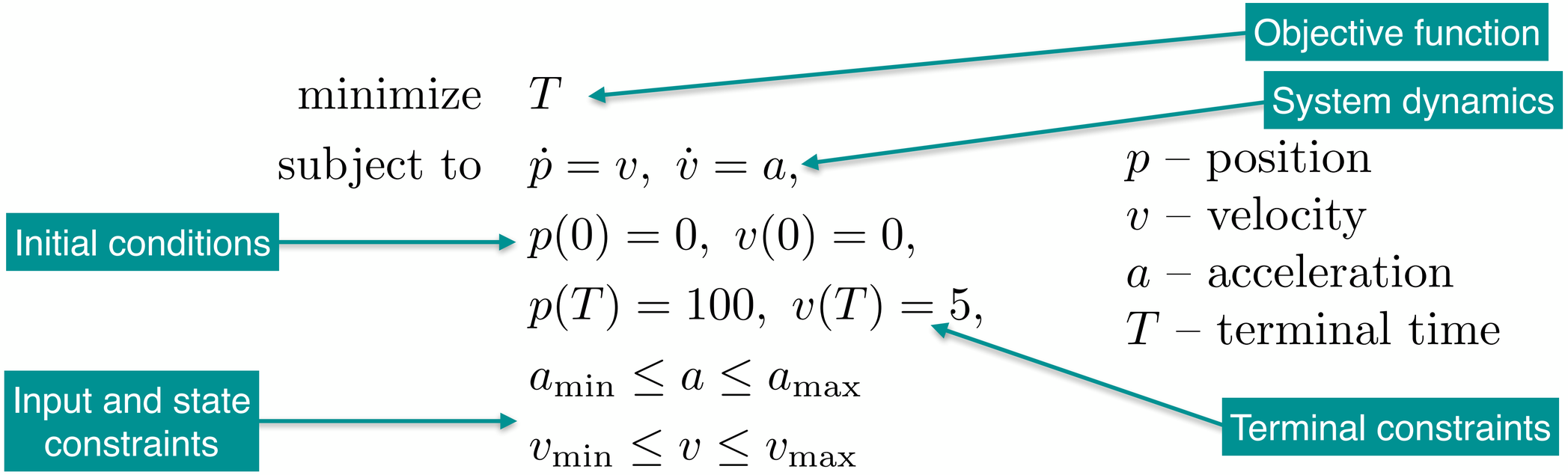
- From A to B in minimum time with velocity and acceleration constraints



- The quantity that we can control on the car is the acceleration a.
- **Solution**: accelerate as fast as possible to $v = v_{max}$ and time deceleration such that velocity at the end matches specification
- Add constraints on comfort, e.g., $|\dot{a}| < a_{max}$, then not so simple ($\dot{a}$ — *jerk*).

A First Optimization Example (2/3)

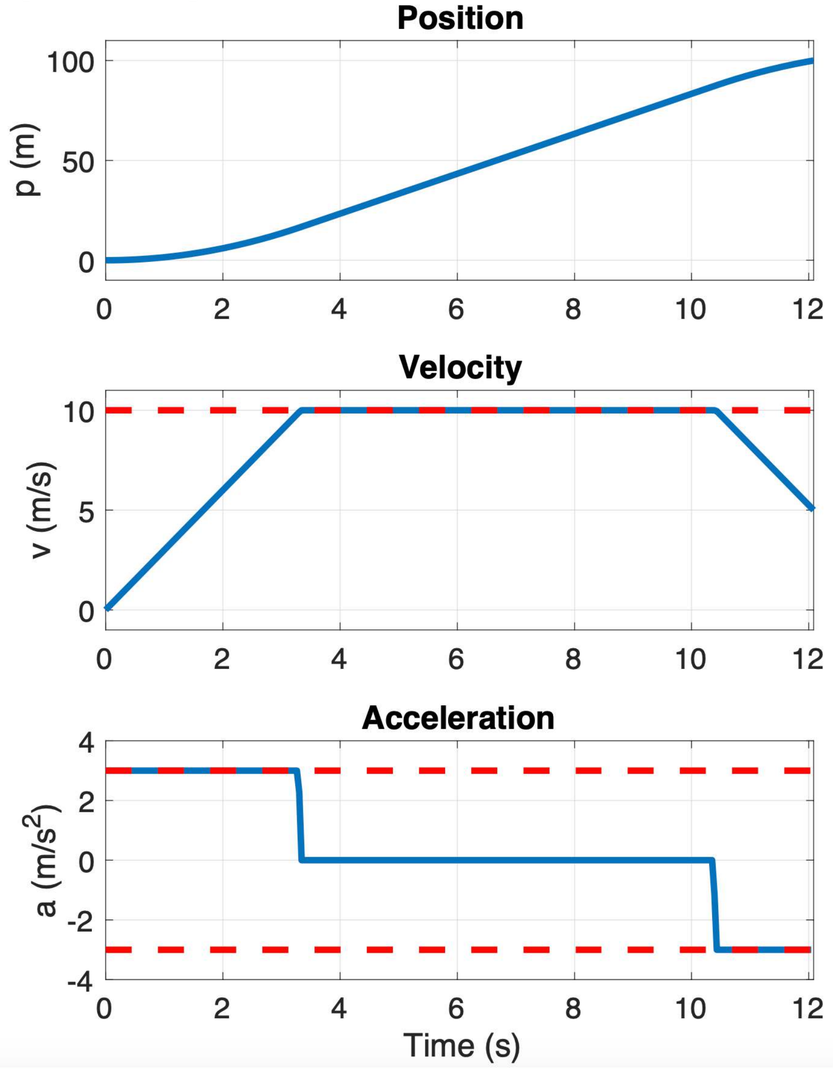
- Mathematical formulation of the motion-planning problem for the autonomous car over time horizon $[0,\ T]$



A First Optimization Example (3/3)

- Resulting trajectories for the position, velocity, and acceleration of the car.

$$
\begin{aligned}
p(0) &= 0,\ p(T) = 100\ \mathrm{m},\\
v(0) &= 0,\ v(T) = 5\ \mathrm{m/s},\\
v_{\min} &= -10\ \mathrm{m/s},\\
v_{\max} &= 10\ \mathrm{m/s},\\
a_{\min} &= -3\ \mathrm{m/s^2},\\
a_{\max} &= 3\ \mathrm{m/s^2}
\end{aligned}
$$



A general description — System Dynamics

- The motion equations, described by differential equations (or difference equations in discrete time), see Lecture 3.
- Often an ODE description

$$\dot{x} = f(t, x, u)$$

t – time
x – states
u – inputs

- It is possible to extend to formulations with algebraic variables and equations (DAE).

A general description — Objective Function

- The function to be optimized (i.e., minimized or maximized) is referred to as the objective function, loss function, cost function

$$J = \int_0^T L(x(t), u(t))\, dt + \Gamma(x(T))$$

- If objective function independent of optimization variables: feasibility problem.
- Can involve optimization variables at any time point in the considered interval $[0,T]$, e.g., integral of quadratic function and penalty on terminal states (at time T).
- Minimize time: $L(x(t), u(t)) = 1$ and $\Gamma(x(T)) = 0$, i.e.,

$$J = \min \int_0^T 1\, dt$$

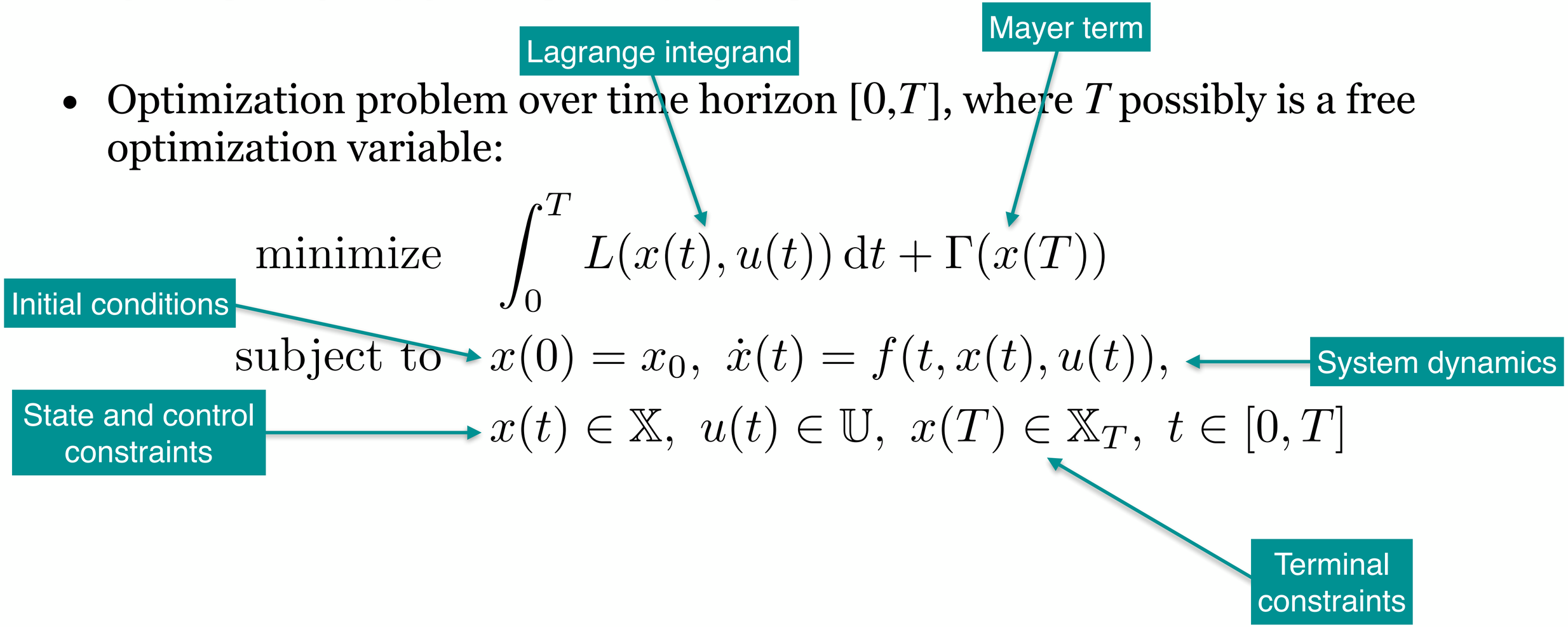
A general description — Constraints

- In addition to the motion equations, there are often several other limitations or requirements to consider.
- Examples:
  - Limits on control signals, such as maximum acceleration.
  - Geometric constraints for vehicle obstacle avoidance.
  - Physical constraints on internal states and other variables, such as maximum power from motor.
- Mathematically described by equalities or inequalities

$$g(x) = 0, \quad h(x) \leq 0, \quad x(t) \in \mathbb{X}(t)$$

Mathematical Formulation

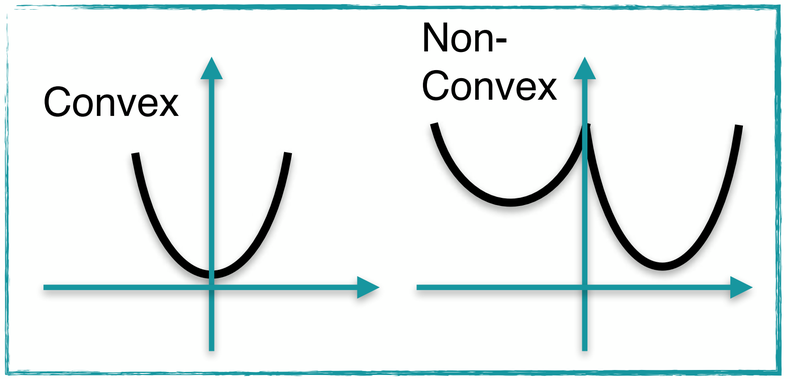
- Optimization problem over time horizon $[0,T]$, where T possibly is a free optimization variable:



Challenges and Solution Strategies for Dynamic Optimization

Challenges (1/2)

- Objective function with equality and inequality constraints.
- Optimization algorithm often finds loopholes in model.
- Continuous-time dynamics (infinite dimensional).
- Often non-linear and non-convex problems in applications.



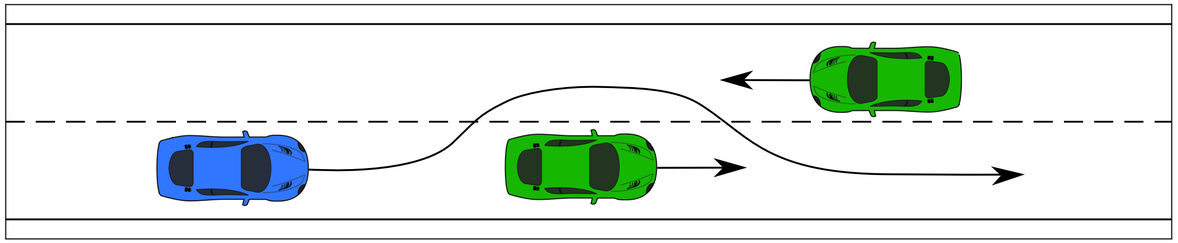
> ”*In constrained as well as in unconstrained minimization, convexity is a watershed concept. The distinction between problems of ’convex’ and ’nonconvex’ type is much more significant in optimization than that between problems of ’linear’ and ’nonlinear’ type.*”
> – R. T. Rockafellar, Fundamentals of Optimization, Univ. of Washington, Seattle.

- Local and global optima of the optimization problem.

Boyd, S. & L. Vandenberghe: “*Convex Optimization*”, Cambridge University Press, 2004.

Challenges (2/2)

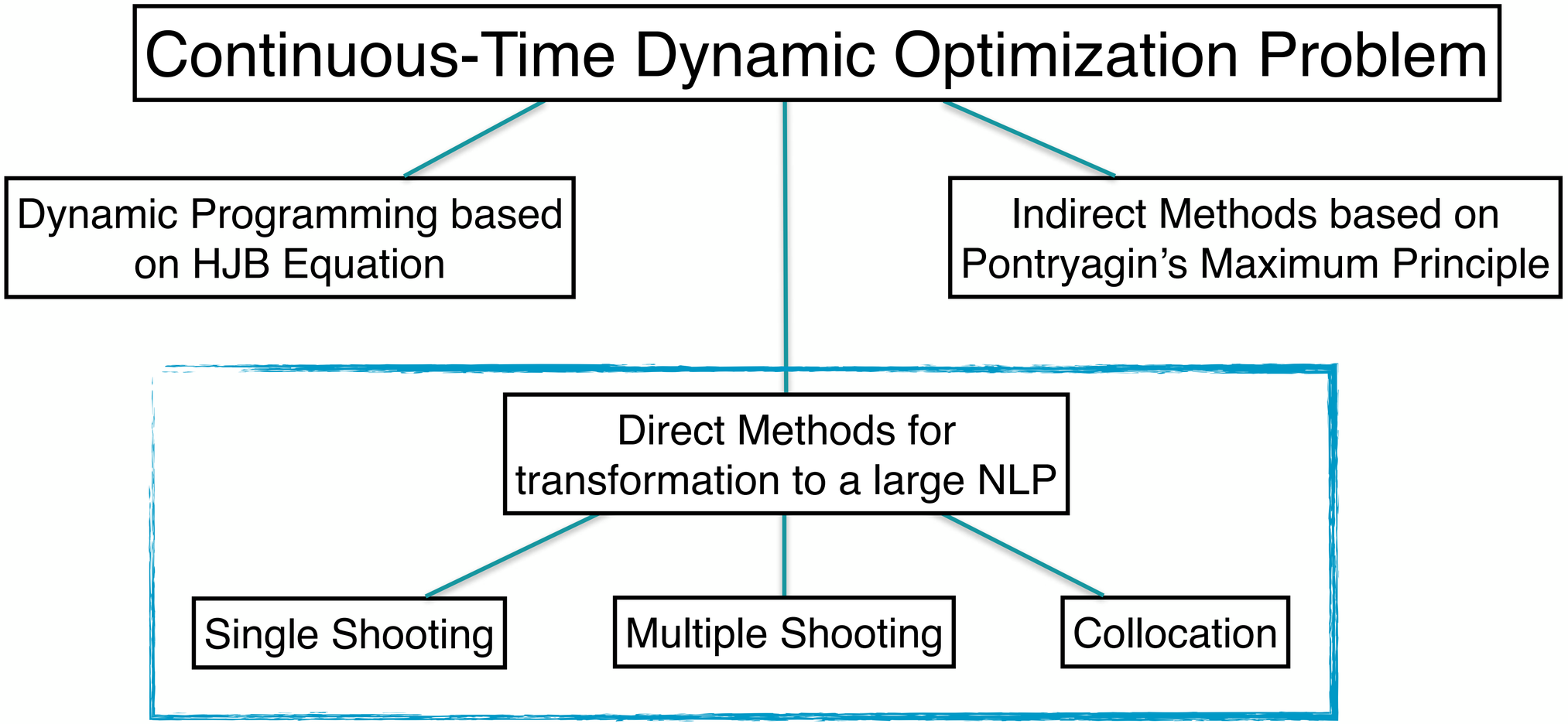
- Convex optimization problem (convex feasible set and convex objective function): a local minimum is also a global minimum.
- Non-convex optimization problem: can be challenging to even find one local optimum
- Very common



- How to find local minima?
  - Recall from previous courses in calculus that local optima of a function can be found using the derivative.
  - Here we mainly consider iterations based on Newton's method and optimality conditions.
  - How to numerically compute required derivatives of involved functions with sufficient accuracy?

Boyd, S. & L. Vandenberghe: "*Convex Optimization*", Cambridge University Press, 2004.

Solution Strategies – Overview



Diehl, M: "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

Numerical Solution of an Optimal Control Problem

- Two major approaches to discretization related to optimization problems:
  - Discretize the control inputs and reformulate optimization problem in initial state and control inputs (sequential).
  - Discretize both control inputs and states and keep all variables in the optimization (simultaneous).

Diehl, M: "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

Direct Methods for Dynamic Optimization

Direct Methods – The Overall Idea

$$\begin{aligned} \text{minimize} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\ \text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)), \\ & x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T,\ t \in [0, T] \end{aligned}$$

Infinite-dimensional optimal control problem

Discretization

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

Optimization problem with a finite set of variables, a non-linear program

Diehl, M: "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

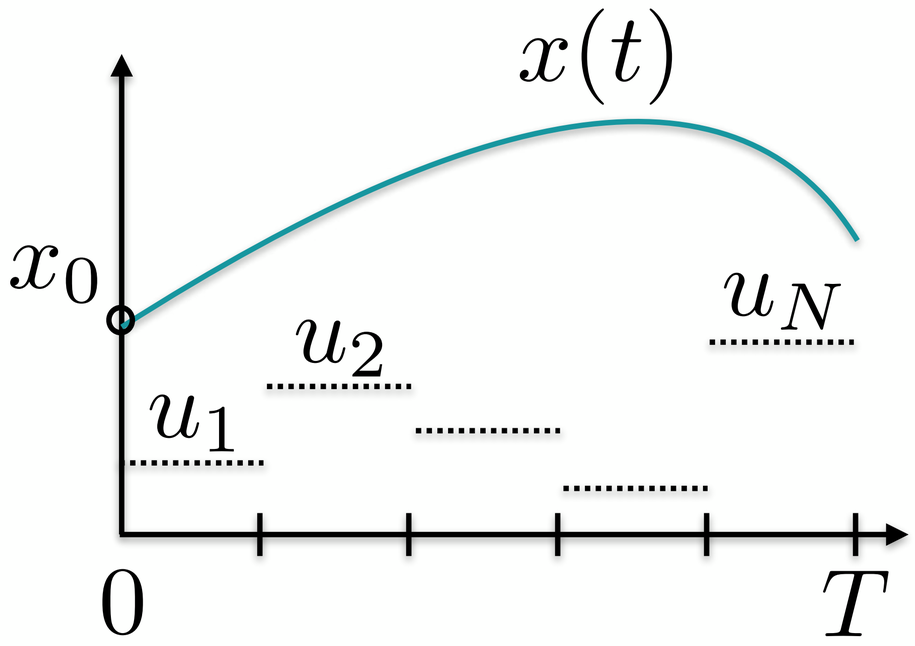
Direct Methods

- Discretize the optimization problem to obtain a a general non-linear programming (NLP) problem
- The (typically large) NLP typically has significant sparsity.
- Focus on direct simultaneous methods in this lecture (well-proven track record in many applications).

Direct Single Shooting (1/2)

- Basic idea of single shooting:
  - Discretize control inputs (piecewise constant in the figure to the right).
  - Start with an initial guess of control inputs and integrate dynamics forward in time with these inputs.
  - Iteratively update control inputs and re-simulate dynamics forward.

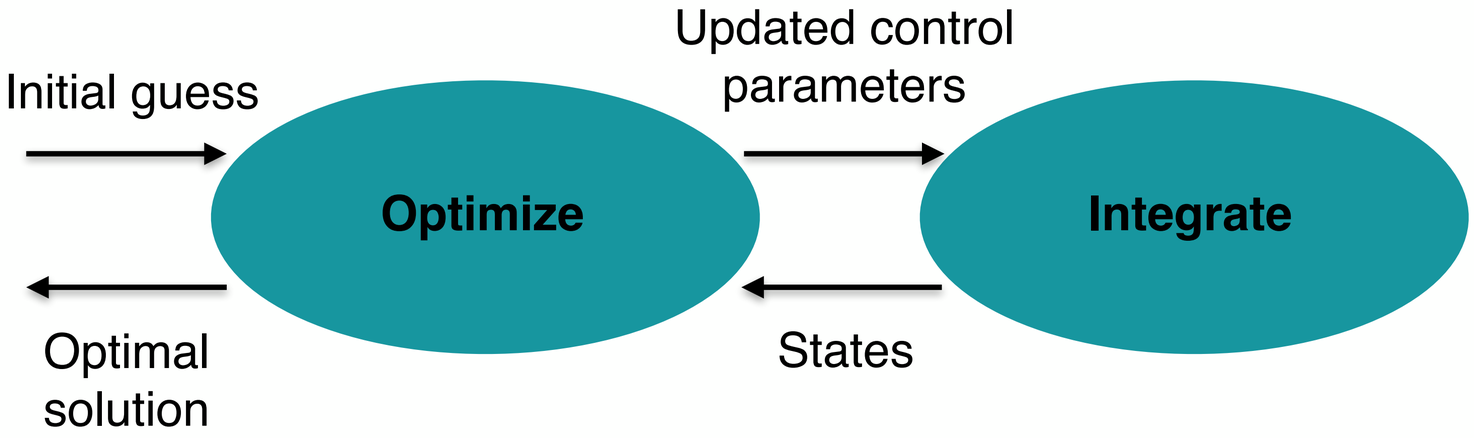
$$\begin{aligned} \underset{u}{\text{minimize}} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\ \text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)) \\ & x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T, \end{aligned}$$



Rawlings, J. B., D. Q. Mayne, & M. Diehl:
"*Model Predictive Control: Theory, Computation, and Design*". 2nd Edition. Nob Hill Publishing, 2017.

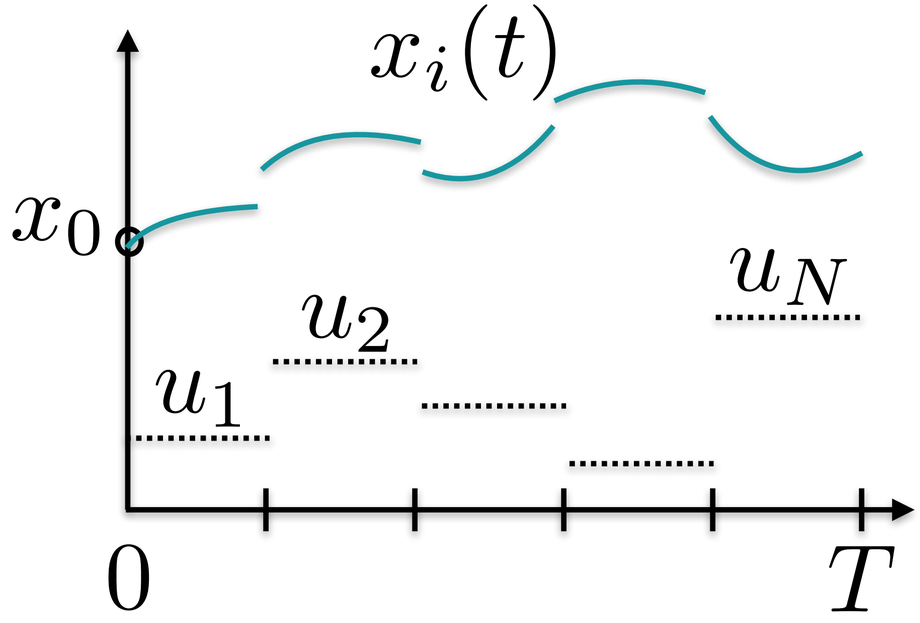
Direct Single Shooting (2/2)

- Sequential approach that optimizes over the control parameters (typically assuming piecewise constant control).
- One of the most straightforward methods to implement
- challenging with unstable system dynamics and handling of state constraints.
- Convergence issues
- high quality initial guess often important



Direct Multiple Shooting (1/2)

- Extension of single shooting:
  - Divide the time horizon into elements and apply single shooting in each element.
  - Add continuity constraints (equality) at the element junctions for the states.
- Optimization over u_i and the continuity variables
- Integration over much shorter time-interval which helps convergence



Direct Multiple Shooting (2/2)

- Explicit Runge-Kutta (RK) methods common for the integration of the states in each element.
- The division into elements provides better numerical accuracy (especially for unstable system dynamics).
- Allows initialization of state trajectories and easier handling of path constraints.
- The number of variables in the optimization problem grows, not only the control signals but also the state-variables
- Sparsity in the Jacobian and Hessian matrices in the resulting NLP can be utilized.

Equality constraints with dots to algebraic constraints

- **Q**: How to transform $\dot{x} = f(x)$ to algebraic constraints $g(x) = 0$?
- **A**: Use techniques from classical ODE integration
- Below, first-order expressions

$$\begin{aligned} \text{minimize} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\ \text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)), \\ & x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T,\ t \in [0, T] \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

Explicit 1st order (Forward Euler)

$$\frac{x_{t+1} - x_t}{h} = f(x_t, u_t)$$

and then the equality constraint is

$$x_{t+1} = x_t + h f(x_t, u_t)$$

Implicit 1st order (Backward Euler)

$$\frac{x_{t+1} - x_t}{h} = f(x_{t+1}, u_{t+1})$$

and the corresponding constraint is

$$x_{t+1} = x_t + h f(x_{t+1}, u_{t+1})$$

Higher order methods directly applicable

- Consider an explicit ODE:

$$\dot{x} = f(t, x)$$

- Numerical integration equations for explicit or implicit Runge-Kutta methods

Explicit

$$\begin{aligned}
x_{n+1} &= x_n + h\sum_{i=1}^{s} b_i k_i, \\
k_1 &= f(t_n, x_n), \\
k_2 &= f(t_n + c_2 h, x_n + h(a_{2,1}k_1)), \\
&\vdots \\
k_s &= f(t_n + c_s h, x_n + h(a_{s,1}k_1 + \ldots + a_{s,s-1}k_{s-1}))
\end{aligned}$$

Implicit

$$\begin{aligned}
x_{n+1} &= x_n + h\sum_{i=1}^{s} b_i k_i, \\
k_1 &= f(t_n + c_1 h, x_n + h(a_{1,1}k_1 + \ldots + a_{1,s}k_s)), \\
k_2 &= f(t_n + c_2 h, x_n + h(a_{2,1}k_1 + \ldots + a_{2,s}k_s)), \\
&\vdots \\
k_s &= f(t_n + c_s h, x_n + h(a_{s,1}k_1 + \ldots + a_{s,s}k_s))
\end{aligned}$$

Reminder – The Overall Idea

$$
\begin{aligned}
\text{minimize} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\
\text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)), \\
& x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T,\ t \in [0, T]
\end{aligned}
$$

Infinite-dimensional optimal control problem

Discretization

$$
\begin{aligned}
\text{minimize} \quad & f(x) \\
\text{subject to} \quad & g(x) = 0, \\
& h(x) \leq 0
\end{aligned}
$$

Optimization problem with a finite set of variables, a non-linear program

Diehl, M: "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

Solving the Non-Linear Program

Non-Linear Program (NLP)

- The direct methods for discretization result in a (typically large) non-linear program (NLP) on the format:

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

- This NLP can be solved using various methods, e.g., interior point (IP), sequential quadratic programming (SQP), ...

Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.

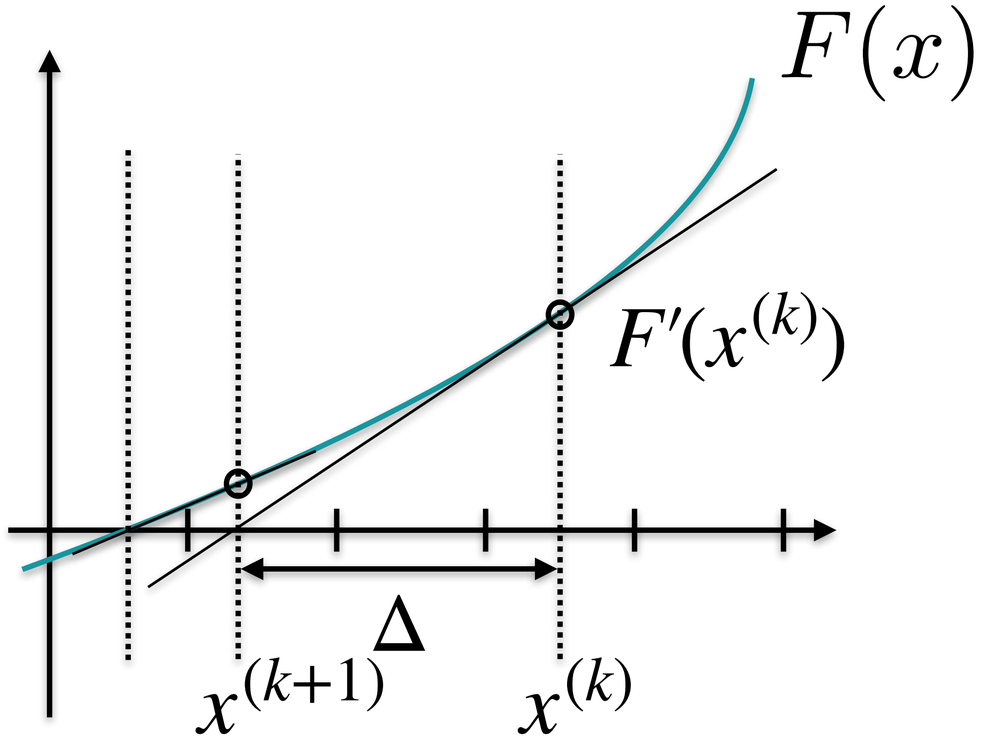
Optimization methods

- This is not a course in optimization but choosing the right solver for the right problem can be essential for performance
  - wrong choice can easily reduce performance orders of magnitude
- Convex vs. non-convex problems
- Common methods in dynamic optimization
  - QP - Quadratic Program
  - SQP - Solve the optimization problem by a sequence of QP
  - Interior point methods - move constraints to the objective via barrier functions
  - … and so many more
- Take-away here: keep in mind that method matters and look it up when you need to

Background: Newton's Method

- **Iterative method** for finding a root x of $F(x)$ based on a starting point.
- Finds **local optimum** for optimization problem – **initialization strategies** for variables important.

1. Given guess $x^{(k)}$:
   $F(x^{(k)} + \Delta) = F(x^{(k)}) + F'(x^{(k)})\Delta + \mathcal{O}(\Delta^2)$
2. Choose update Δ such that
   $F(x^{(k)}) + F_x(x^{(k)})\Delta = 0$
3. $x^{(k+1)} = x^{(k)} + \Delta$ and iterate until convergence



Automatic Differentiation (AD)
- compute derivatives efficiently and with ease

- Derivatives are useful and difference approximations are expensive and inaccurate. AD a structured way of computing derivatives with machine precision.
- Chain rule for differentiation used to decompose the problem into smaller elementary operations.
- Provides Jacobians (first-order derivatives) and Hessians (second-order derivatives) for solution of the NLP using Newton-based methods.
- Forward or backward mode can give very different performance.
  - Forward mode when #inputs $\ll$ #outputs.
  - Backward mode when #inputs $\gg$ #outputs.
- Example: backpropagation in training of neural networks.

Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.

Automatic Differentiation – Example

- Compute gradient of F using AD in forward mode:

$$F(x_1, x_2) = \cos(x_1) + x_1 x_2$$

- Decompose into elementary operations and keep track of derivatives:

$x_3 = x_1 x_2$	$x_3' = x_1' x_2 + x_1 x_2'$
$x_4 = \cos(x_1)$	$x_4' = -\sin(x_1)\, x_1'$
$x_5 = x_3 + x_4$	$x_5' = x_3' + x_4'$

- The final row in the right column is the desired derivative. Performing these computations twice for the so called seeds $(x_1', x_2') = (1,0)$ and $(x_1', x_2') = (0,1)$ respectively gives the desired gradient

$$\nabla F(x_1, x_2) = (F_{x_1}(x_1, x_2), F_{x_2}(x_1, x_2))$$

- Only one sweep using backward mode AD.

Optimality Conditions

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

- Introduce the Lagrangian function:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \lambda^{\mathrm{T}} g(x) + \nu^{\mathrm{T}} h(x)$$

- First-order optimality conditions, given certain technical conditions on the constraints (constraint qualification), by Karush, Kuhn, and Tucker (KKT):

$$\begin{aligned} \nabla_x \mathcal{L}(x^*, \lambda^*, \nu^*) &= 0, && \leftarrow \text{Stationarity} \\ g(x^*) &= 0, && \leftarrow \text{Feasibility} \\ h(x^*) &\leq 0, && \leftarrow \text{Feasibility} \\ \nu^* &\geq 0, && \leftarrow \text{Complementarity} \\ \nu_i^* h_i(x^*) &= 0, \quad i = 1, \ldots, n_h && \leftarrow \text{Complementarity} \end{aligned}$$

Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.

Solution of NLP (1/2)

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

- For **equality-constrained NLPs,** i.e., no $h(x)$, the KKT conditions give a nonlinear system of equations to be solved:

$$\begin{pmatrix} \nabla_x \mathcal{L}(x, \lambda) \\ g(x) \end{pmatrix} = 0$$

- Apply **Newton's method** with variables and function:

$$\tilde{x} = \begin{pmatrix} x \\ \lambda \end{pmatrix}, \quad F(\tilde{x}) = \begin{pmatrix} \nabla_x \mathcal{L}(x, \lambda) \\ g(x) \end{pmatrix}$$

Diehl, M: "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

Solution of NLP (2/2)

- Application of Newton's method for the case with no $h(x)$ gives the iterations as the solution of the linear equation system:

$$\begin{pmatrix} \nabla_x \mathcal{L}(x_k, \lambda_k) \\ g(x_k) \end{pmatrix} + \begin{pmatrix} \nabla_x^2 \mathcal{L}(x_k, \lambda_k) & \nabla g(x_k) \\ \nabla g(x_k)^{\mathrm{T}} & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta \lambda \end{pmatrix} = 0$$

- Requires the Hessian of the Lagrangian function.
- Partial Newton step with line-search or trust-region strategies to ensure intended decrease of specified measure.
- Quasi-Newton methods with approximate Hessian exist (of which BFGS is one of the most common).

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

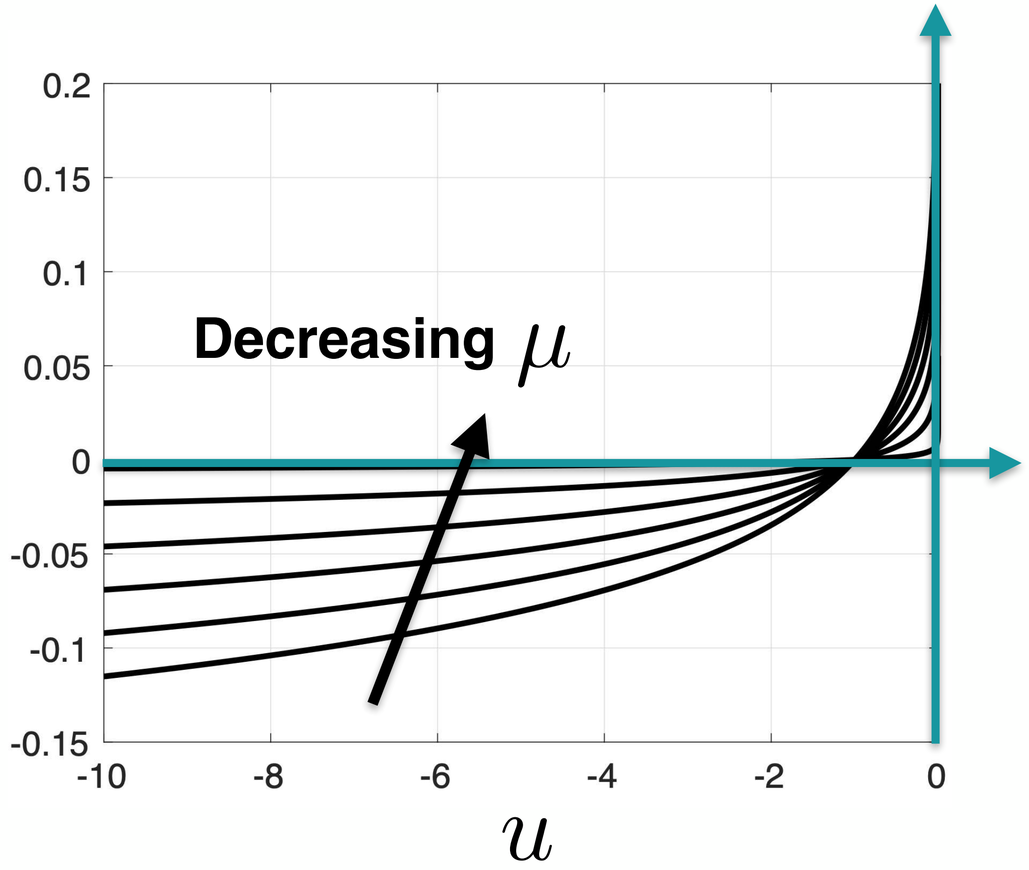
Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.

Move inequalities to the loss-function — barrier functions

- Consider the following approximation of a barrier function:

$$-\mu \log(-u)$$

- The approximation of the barrier improves for decreasing values of the parameter μ.



Interior-Point Methods (1/2)

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

- Consider equality and inequality-constrained NLP problems.
- Interior-point methods with barrier functions for inequalities – move inequality constraints to objective function with barrier function and positive parameter μ:

$$\begin{aligned} \text{minimize} \quad & f(x) + \mu \sum_{i=1}^{n_h} \log(-h_i(x)) \\ \text{subject to} \quad & g(x) = 0 \end{aligned}$$

- Could then be solved as an equality-constrained problem
- IPOPT is a state-of-the-art implementation of such kind of NLP solver.

Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.

Sequential Quadratic Programming

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

- Alternative to IP methods:
  Sequential quadratic programming (SQP).
- **Iteratively** applies a linearization to the inequality constraints and the equality constraints around the current solution to obtain the quadratic program (QP):

$$\begin{aligned} \text{minimize} \quad & \nabla f(x_k)^{\mathrm{T}}(x - x_k) + \frac{1}{2}(x - x_k)^{\mathrm{T}} \nabla_x^2 \mathcal{L}(x_k, \lambda_k, \nu_k)(x - x_k) \\ \text{subject to} \quad & g(x_k) + \nabla g(x_k)^{\mathrm{T}}(x - x_k) = 0, \\ & h(x_k) + \nabla h(x_k)^{\mathrm{T}}(x - x_k) \leq 0 \end{aligned}$$

- QP can be solved using **IP methods** or **active set methods**.

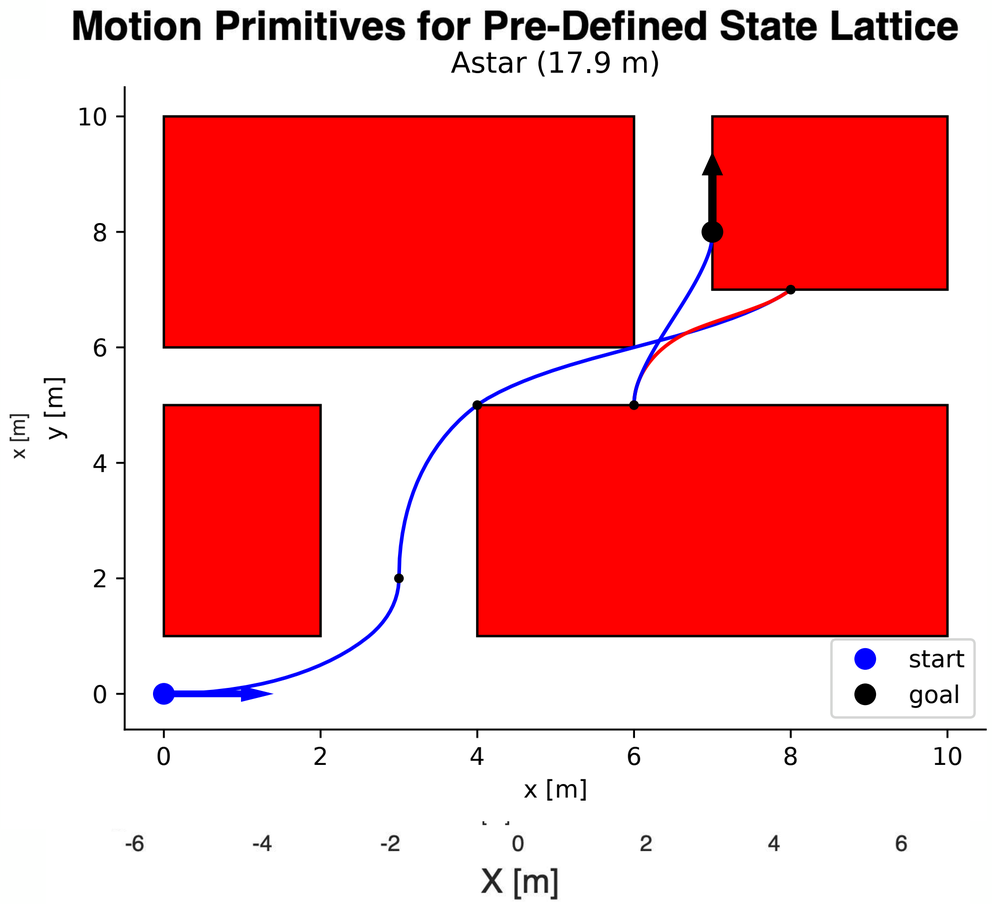
Nocedal, J., & S. Wright, "*Numerical Optimization*". Springer, 2006.
Diehl, M., "*Numerical Optimal Control*", Optec, K.U. Leuven, Belgium, 2011.

A Small Use-Case and Implementation

Case Study: Optimization of Motion Primitives

- Optimization can be used to compute the motion primitives from Lecture 4 (and Hand-in 2)
- Example code using the tool CasADi (py and m-files on course git)
- Vehicle motion equations given by:

$$\dot{x} = v\cos(\theta),$$
$$\dot{y} = v\sin(\theta),$$
$$\dot{\theta} = v/L\tan(u)$$



A Small Problem

- Let the state $x(t) = (X(t), Y(t), \theta(t))$ with the motion model

$$\dot{X} = v\ \cos(\theta)$$

$$\dot{Y} = v\ \sin(\theta)$$

$$\dot{\theta} = v/L\ \tan u$$

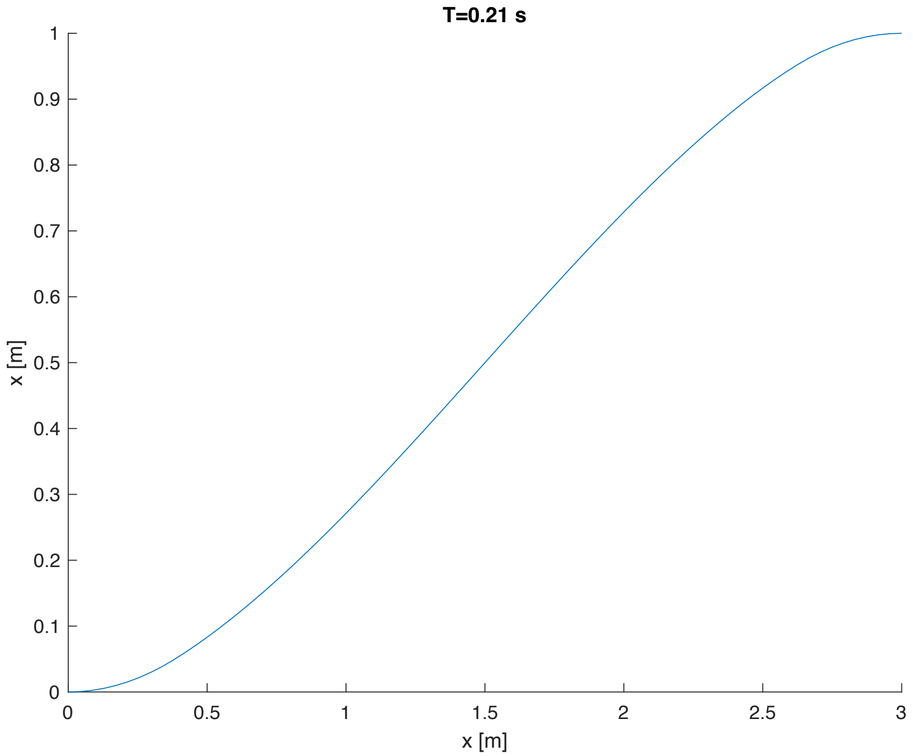
- The problem is defined as

$$\underset{u(t),x(t)}{\text{minimize}} \quad T$$

$$\text{such that} \quad x(0) = x_i, x(T) = x_f$$

$$\dot{x} = f(x(t), u(t))$$

$$|u(t)| \leq u_{\max}$$



Dynamic Problem to NLP

$$\begin{aligned} \underset{u(t),x(t)}{\text{minimize}} \quad & T \\ \text{such that} \quad & x(0) = x_i, x(T) = x_f \\ & \dot{x} = f(x(t), u(t)) \\ & |u(t)| \leq u_{\max} \end{aligned}$$

$$\begin{aligned} \underset{u_0,\ldots,u_N,x_0,\ldots,x_{N+1}}{\text{minimize}} \quad & Nh \\ \text{such that} \quad & x_0 = x_i, x_{N+1} = x_f \\ & x_{k+1} = x_k + hf(x_k, u_k) \\ & |u_k| \leq u_{\max} \\ & k = 0, \ldots, N \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\ \text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)), \\ & x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T,\ t \in [0, T] \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & g(x) = 0, \\ & h(x) \leq 0 \end{aligned}$$

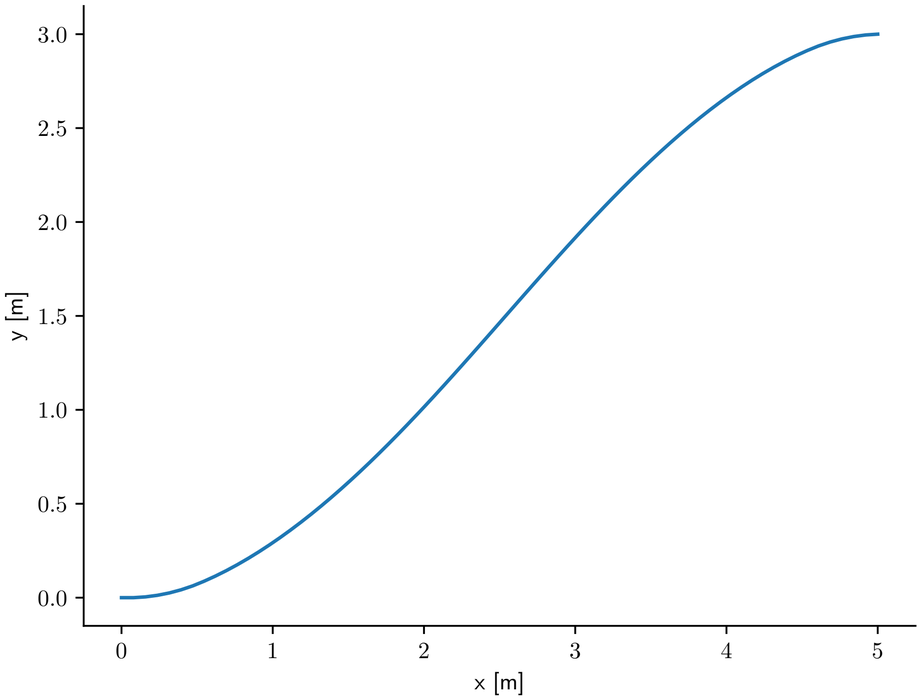
Implementation in CasADi

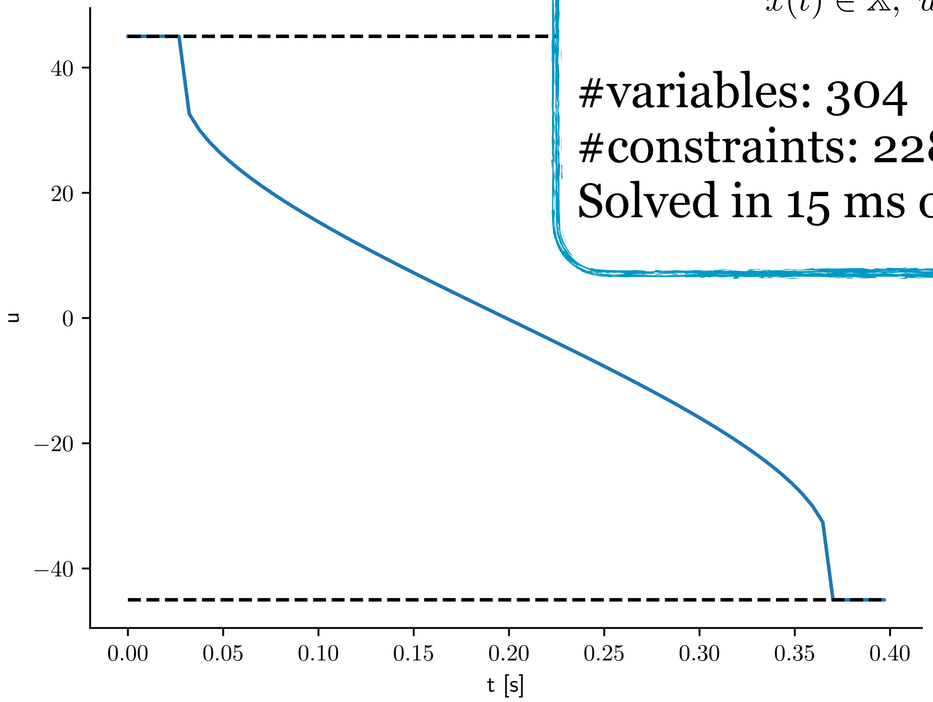
```python
import casadi
import numpy as np


# Vehicle parameters and constraints
L = 1.5  # Wheel base (m)
v = 15  # Constant velocity (m/s)
u_max = np.pi / 4  # Maximum steering angle (rad)

state_i = [0, 0, 0]  # Initial state
state_f = [10, 3, 0]  # Final state

# %% Define optimization variables and motion equations
x = casadi.MX.sym("x", nx)
u = casadi.MX.sym("u")

F = casadi.Function("f", [x, u], [v * casadi.cos(x[2]), v * casadi.sin(x[2]), v * casadi.tan(u) / L])

# Create optimizer object
opti = casadi.Opti()
```

Implementation in CasADi

```
N = 75  # Number of elements

X = opti.variable(nx, N + 1)
U = opti.variable(1, N)
T = opti.variable(1)

opti.minimize(T)

# Set initial guess values of variables
opti.set_initial(T, 0.1)
opti.set_initial(U, 0.0 * np.ones(N))
opti.set_initial(pos_x, np.linspace(state_i[0], state_f[0], N + 1))
opti.set_initial(pos_y, np.linspace(state_i[1], state_f[1], N + 1))

# Constrain initial and final state
opti.subject_to(X[:, 0] == state_i)
opti.subject_to(X[:, -1] == state_f)

# Add motion and steer signal constraints
dt = T / N
for k in range(N):  # Euler-forward integration equations
    opti.subject_to(X[:, k + 1] == X[:, k] + dt * casadi.vertcat(*F(X[:, k], U[:, k])))
    opti.subject_to(U[k] <= u_max)
    opti.subject_to(-u_max <= U[k])

# Solve
opti.solver("ipopt")
sol = opti.solve()
```

Solution





$$
\begin{aligned}
\text{minimize} \quad & \int_0^T L(x(t), u(t))\,\mathrm{d}t + \Gamma(x(T)) \\
\text{subject to} \quad & x(0) = x_0,\ \dot{x}(t) = f(t, x(t), u(t)), \\
& x(t) \in \mathbb{X},\ u(t) \in \mathbb{U},\ x(T) \in \mathbb{X}_T,\ t \in [0, T]
\end{aligned}
$$

#variables: 304
#constraints: 228
Solved in 15 ms on my machine

Code in Python and Matlab available in course git-repo

References and Further Reading

References and Further Reading

Some sources if youo want to know more

- Limebeer, D. J., & A. V. Rao: ”*Faster, higher, and greener: Vehicular optimal control*”. IEEE Control Systems Magazine, 35(2), 36-56, 2015.
- Andersson, J., J. Gillis, G. Horn, and J. B. Rawlings, & M. Diehl: ”*CasADi–A software framework for nonlinear optimization and optimal control*”, Mathematical Programming Computation, 2018.
- Diehl, M, “*Numerical Optimal Control*”, Optec, K.U. Leuven, Belgium, 2011.
- General optimization litterature
  - Boyd, S. & L. Vandenberghe, “*Convex Optimization*”. Cambridge University Press, 2004.
  - Nocedal, J., & S. Wright, “*Numerical Optimization*”. Springer, 2006.

Take home messages

- General approach for optimal trajectory planning
  - Very useful in Model Predictive Control, see upcoming lecture
- There are algorithms for solving dynamic optimization problems involving motion equations
- Key notions: direct methods, single/multiple shooting, collocation
- For direct methods:
  - Transform dynamic optimization problem to a large non-linear programming (NLP) problem by discretization in different ways.
  - Key step in many solvers — classical Newton-iterationer
  - Efficiency, handling of inequalities — state-of-the-art solvers
- Automatic differentiation — very useful for computing derivatives

www.liu.se

LINKÖPINGS UNIVERSITET